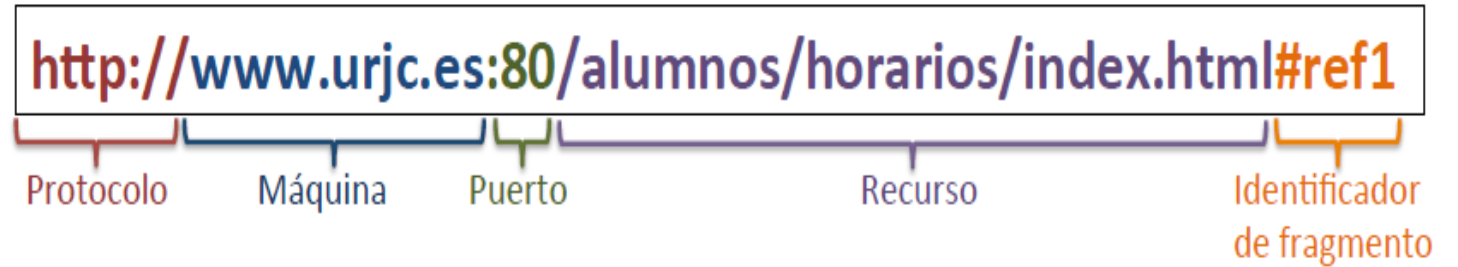


INTRODUCCIÓN A REDES



URL



Protocolo: Protocolo por el que se accede al recurso. Por defecto el predeterminado para la aplicación que usa la URL

Máquina: Maquina en la que reside el recurso. Por defecto la maquina local.

Puerto: Puerto de la maquina a través del que se pide el recurso. Por defecto el predeterminado para el protocolo (http=80)

Recurso: Identificación del recurso dentro de la máquina, incluyendo (a veces) un path. Por defecto el recurso predeterminado para la máquina

Identificador del fragmento: Opcionalmente, se utiliza para identificar un fragmento del recurso

```
<!DOCTYPE html>
<html>
<head>
<title>Mi primera pagina HTML</title>
</head>

<body>
<h1>Primer Título</h1>

<p>
<a href="http://www.w3schools.com">Esto es un enlace</a>
</p>

<h2>Primer Subtítulo</h2>

<p>
<!-- Esto es un comentario -->
Una imagen:

</p>

<p>
<!-- Imagen con URL completa por si viene de otro directorio y/o servidor -->

</p>

<p>
Si en una URL no se especifica servidor, se entiende que es el
mismo. Idem para el directorio</p>
</body>
</html>
```

HTML

Primer Título

[Esto es un enlace](#)

Primer Subtítulo

Una imagen:



Si en una URL no se especifica servidor, se entiende que es el mismo. Idem para el directorio

HTTP

- Protocolo de nivel de aplicación utilizado para transferir recursos hipermedia entre ordenadores.
- Sigue el **modelo Cliente-Servidor**:
 - **Cliente HTTP**: navegador web que pide páginas y, al recibirlas, las muestra al usuario. Ej.: Firefox, Explorer, Chrome, Safari. . .
 - **Servidor HTTP**: servidor web en el que están alojadas paginas que piden los clientes. Ej.: Apache, IIS. . .
- Funciona sobre TCP como protocolo de transporte
- Por defecto un servidor HTTP escucha en el puerto 80, pero puede usar cualquier otro puerto.
- HTTP puede servir tanto **contenido estático** (ficheros) como **contenido dinámico** (el resultado de ejecutar programas en el servidor).

VERSIONES HTTP

0.9: Primera versión documentada, no tiene número de versión oficial, pero es referida como versión 0.9 (1991)

1.0: Primera versión oficial (RFC-1945, año 1996)

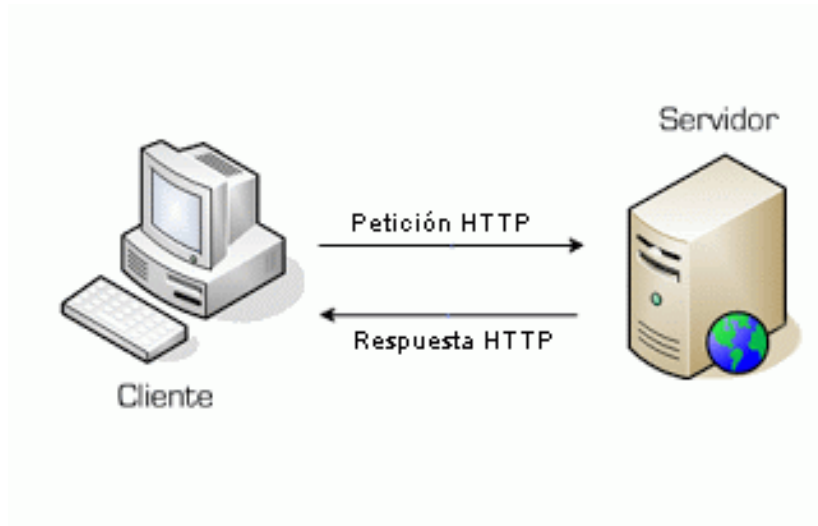
1.1: RFC-2068, año 1997 y RFC-2616, año 1999)

2.0: RFC-7540, año 2015

3.0: RFC 9114, versión actual

PÁGINAS WEB

- Una página web se compone de uno o más **recursos**.
- Cada recurso suele ser un archivo (un archivo HTML, una foto PNG, un applet Java, etc) al que se hace referencia a través de su URL.
- La mayoría de las páginas web están formadas por un archivo HTML base y diversos recursos referenciados dentro de ese archivo base como contenido adicional de la misma página.
 - Ej.: Una página web puede estar compuesta por 6 recursos: 1 fichero HTML y 5 imágenes PNG.



- Pasos:
 1. El cliente abre una conexión TCP con el servidor.
 2. El cliente envía un mensaje de petición HTTP.
 3. El servidor responde con un mensaje de respuesta HTTP.
 4. El servidor cierra la conexión TCP.
- El protocolo HTTP **no mantiene estado**: un servidor trata cada petición HTTP de manera aislada, sin almacenar información sobre las peticiones previas hechas por un mismo cliente.
 - NOTA: con frecuencia las aplicaciones web necesitan guardar estado, aunque el protocolo no lo guarde.

INTERACCIÓN CLIENTE-SERVIDOR EN HTTP

- Supongamos que un navegador solicita la URL:
<http://www.uni.edu/dept/index.html>
que consta de 1 fichero de texto HTML y 3 imágenes PNG

1a. El cliente HTTP inicia la conexión TCP con el servidor HTTP de www.uni.edu en el puerto 80.

2. El cliente HTTP envía un **mensaje HTTP de petición** a través de la conexión TCP. El mensaje indica que el cliente quiere obtener el recurso [/dept/index.html](http://www.uni.edu/dept/index.html)

1b. El servidor HTTP en www.uni.edu estaba esperando conexiones TCP en el puerto 80 y “acepta” esta conexión

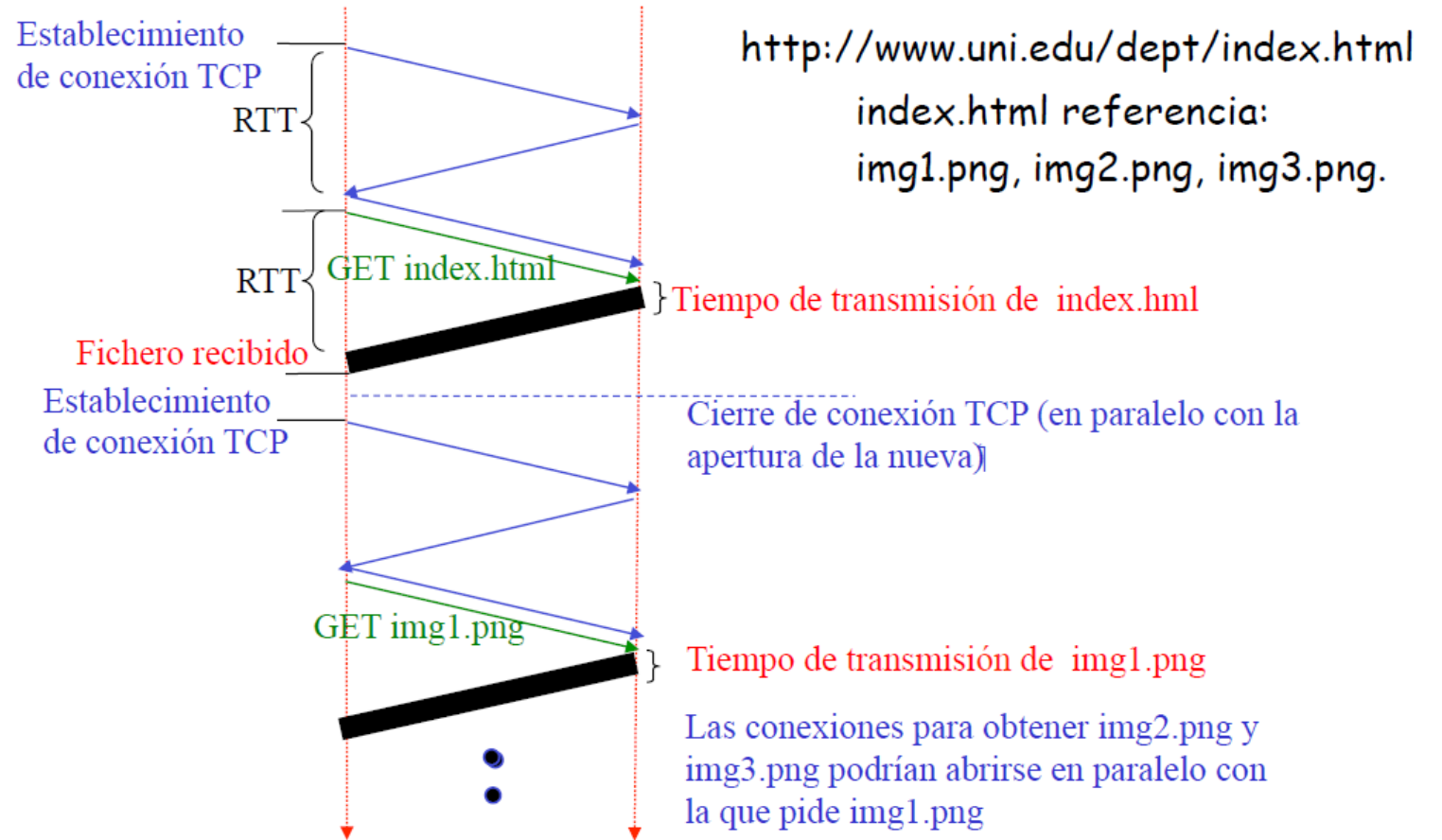
3. El servidor HTTP recibe el mensaje de petición, compone un **mensaje HTTP de respuesta** que contiene el recurso solicitado, y lo envía al cliente.

4. El servidor HTTP cierra la conexión TCP.

5. El cliente HTTP recibe el mensaje de respuesta que contiene el archivo html. Analizando el archivo HTML se encuentran referenciados 3 recursos PNG que forman parte de la página.

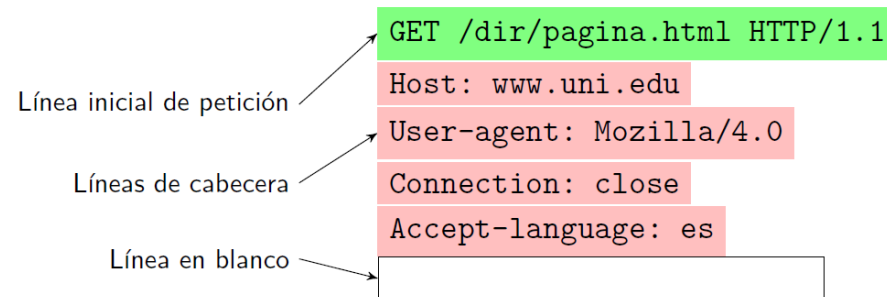
6. Se repiten los pasos del 1 al 4 para cada uno de los 3 recursos PNG.

CONEXIONES NO PERSISTENTES



LÍNEA INICIAL

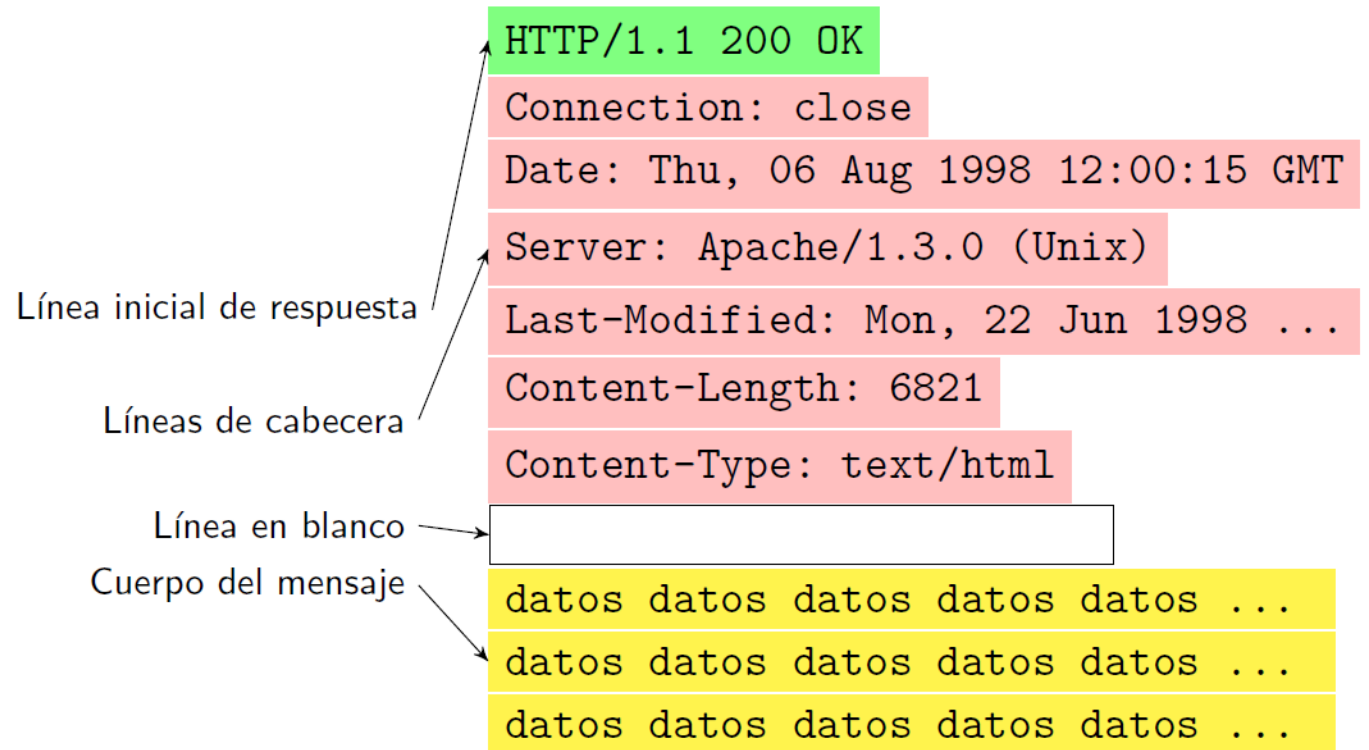
- Especifica el recurso que se solicita y que se quiere de el:
 - Nombre del método (GET, POST, HEAD)
 - Trayecto de acceso (path)
 - Versión de HTTP (en la forma HTTP/s.)
- Ejemplo:
GET /dir/pagina.html HTTP/1.1



LÍNEA INICIAL EN RESPUESTAS

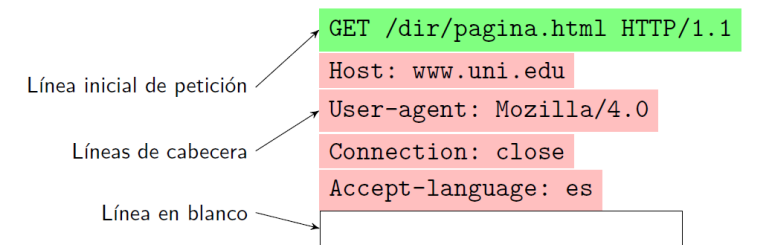
- Proporciona información de estado:
 - Versión de HTTP (siempre HTTP/s.).
 - Código de estado numérico.
 - Código de estado textual.
- Códigos de estado:
 - 1xx: Mensaje informativo.
 - 2xx: Resultado con éxito (Ej.: 200 OK).
 - 3xx: Redirección del cliente a otra URL (Ej.: 301 Moved
Redirección).
 - 4xx: Error en el lado del cliente (Ej.: 404 Not
Found).
 - 5xx: Error en el lado del servidor (Ej.: 500
Server Error).
- Ejemplo:
 - HTTP/1.0 200 OK

EJEMPLO RESPUESTA



LÍNEA CABECERA

- Especifica el recurso que se solicita y que se quiere de el:
 - Nombre del método (GET, POST, HEAD)
 - Trayecto de acceso (path)
 - Versión de HTTP (en la forma HTTP/s.)
- Ejemplo:
 - GET /dir/pagina.html
HTTP/1.1



MÉTODOS

- **GET:**
 - Solicita un recurso al servidor especificando su URL.
- **HEAD:**
 - Igual que un GET, pero solo pide las cabeceras. Con este método se pueden consultar todas las cabeceras sin descargar el recurso completo.
- **POST:**
 - Envía datos al servidor, normalmente los introducidos por el usuario en un formulario.
 - Los datos van en el cuerpo.
 - El path de la línea inicial (URL) se refiere normalmente al programa que tratará los datos que se envían.
 - NOTA: También se pueden enviar datos con un GET. En este caso, los datos van codificados en el path de la línea inicial (URL), y no hay cuerpo.

MÉTODOS

- **PUT:**
 - Actualiza información sobre un recurso del servidor. Similar a POST, pero el servidor debe almacenar en el path que acompaña al método en la línea inicial el contenido del mensaje.
- **DELETE:**
 - Elimina en el servidor el documento especificado.

USANDO HTTP

1. En un terminal, se realiza un telnet al puerto 80 de la máquina del servidor:

```
telnet www.urjc.es 80
```

Abre una conexión TCP con el puerto 80 de www.urjc.es. Cualquier cosa que se teclee se enviara por la conexión.

2. En un terminal, se realiza un telnet al puerto 80 de la maquina del servidor:

```
GET /index.html HTTP/1.0
```

Envía una petición de la página `index.html`. Es necesario dejar una línea en blanco para terminar la cabera HTTP.

3. Se muestra la respuesta recibida del servidor.